

Organización de Trabajo — Fase 1

Trabajo Integrador – Ingeniería de Software

Sistema: Plataforma Predictiva de Producción de Hidrocarburos

1. Objetivo de la Fase 1

El objetivo de la Fase 1 es **construir una demo funcional del sistema** que sirva como base para el desarrollo posterior.

En esta etapa **no se implementa aún el modelo predictivo real**, sino que se desarrolla:

- Una **API REST funcional (mock)** que simule el comportamiento del sistema final.
- Una **infraestructura reproducible** mediante contenedores.
- Un **pipeline de integración continua (CI/CD)**.
- Un **sistema de monitoreo** para observar el comportamiento del servicio.
- La **documentación técnica necesaria** para comprender el diseño del sistema.

La idea es construir una **base de arquitectura sólida** que permita continuar el desarrollo de manera ágil en las siguientes fases.

2. Arquitectura General del Sistema

El sistema estará compuesto por los siguientes componentes principales:

1. **API Backend**
 - Expone endpoints para consultar pronósticos.
 - Implementa la lógica mock del sistema.
2. **Infraestructura y Despliegue**
 - Contenerización con Docker.
 - Pipeline de CI/CD automatizado.
3. **Observabilidad**
 - Recolección de métricas del sistema.
 - Visualización mediante dashboards.

Diagrama conceptual simplificado:

Usuario / Cliente



API REST (Mock Forecast Service)

↓

Generador de datos mock

Sistema de monitoreo:

Prometheus → Grafana

Infraestructura:

Docker + CI/CD

3. Distribución de Responsabilidades

El trabajo se divide en **tres áreas principales**, cada una con un responsable principal.

Sin embargo, **todos los integrantes deben comprender el funcionamiento completo del sistema**.

La división permite avanzar en paralelo y distribuir la carga técnica de manera equilibrada.

Integrante 1 — Backend y API

Responsabilidad principal: **desarrollar el servicio API que simula el sistema predictivo**.

Tareas

Implementación del servidor API

Desarrollar el servidor utilizando un framework backend (ej: FastAPI).

Implementación de endpoints

Endpoints requeridos:

GET /api/v1/forecast

Permite obtener el pronóstico de producción de un pozo.

Parámetros:

- id_well
- date_start
- date_end

GET /api/v1/wells

Devuelve el listado de pozos disponibles.

Parámetro:

- date_query

Implementación de lógica mock

El sistema debe devolver datos simulados, por ejemplo:

- valores constantes
- valores generados con tendencia lineal
- valores aleatorios dentro de un rango

El objetivo es **simular el comportamiento del modelo real que se implementará en fases posteriores.**

Seguridad básica

La API debe validar una API key mediante el header:

X-API-Key: abcdef12345

Si la clave no es válida se debe devolver:

HTTP 403 – Forbidden

Definición de modelos de datos

Definir los schemas de entrada y salida de la API.

Documentación de la API

Utilizar OpenAPI / Swagger para documentar:

- endpoints
- parámetros
- ejemplos de respuesta
- descripciones de cada operación

Resultado esperado:

- API funcional
 - documentación accesible desde Swagger
-

Integrante 2 — Infraestructura y DevOps

Responsabilidad principal: **garantizar que el sistema sea reproducible y desplegable automáticamente.**

Tareas

Contenerización con Docker

Crear un Dockerfile para ejecutar el servicio API dentro de un contenedor.

El contenedor debe incluir:

- runtime
- dependencias
- código fuente

Orquestación de servicios

Crear un archivo docker-compose que permita levantar el sistema completo.

Servicios esperados:

- API
- Prometheus
- Grafana

Comando esperado para levantar el sistema:

`docker compose up`

Pipeline de CI/CD

Configurar un pipeline automático que se ejecute cuando se realiza un commit o pull request.

El pipeline debe realizar:

1. instalación de dependencias
2. ejecución de tests
3. análisis estático de código (si se utiliza)
4. build de imagen Docker

Opcionalmente:

5. publicación de la imagen en un registry

Automatización del despliegue

El pipeline debe permitir desplegar el sistema automáticamente en un entorno de desarrollo.

Resultado esperado:

- proyecto completamente dockerizado
 - pipeline de CI/CD funcional
-

Integrante 3 — Dashboards + Docs

Responsabilidad principal: **monitorear el sistema y documentar decisiones técnicas.**

Tareas

Sistema de monitoreo

Implementar un sistema que recolecte métricas del servicio API.

Herramientas sugeridas:

- Prometheus (recolección de métricas)
- Grafana (visualización)

Métricas a monitorear

El sistema debe monitorear al menos:

- latencia de requests
- cantidad de requests
- tasa de errores
- disponibilidad del servicio

Dashboard

Crear un dashboard que muestre:

- tiempo de respuesta de la API
- volumen de consultas
- estado del servicio

Alertas (opcional)

Configurar alertas básicas ante:

- tasa alta de errores
- latencia elevada

Architecture Decision Records (ADR)

Documentar decisiones importantes del proyecto.

Ejemplos:

ADR-001 — Elección del framework backend

ADR-002 — Uso de Docker para contenerización

ADR-003 — Uso de Prometheus + Grafana para monitoreo

Cada ADR debe explicar:

- problema
- alternativas consideradas
- decisión tomada
- justificación

Resultado esperado:

- sistema de monitoreo funcional
 - documentación de arquitectura
-

4. Flujo de Trabajo del Equipo

El proyecto debe desarrollarse utilizando **buenas prácticas de trabajo colaborativo**.

Estructura del repositorio

project/

api/

código del backend

infra/

docker y configuración de despliegue

monitoring/

configuración de prometheus y grafana

docs/

ADR y documentación técnica

README.md

Flujo de trabajo con Git

1. Crear una rama por funcionalidad.

Ejemplo:

feature/api
feature/docker
feature/monitoring

2. Implementar cambios en la rama correspondiente.
3. Abrir un Pull Request.
4. Otro integrante revisa el código.
5. Realizar ajustes si es necesario.
6. Merge a la rama principal.

Este proceso permite:

- mantener la calidad del código
 - distribuir el trabajo equitativamente
 - dejar registro del aporte de cada integrante.
-

5. Integración entre componentes

El desarrollo se realiza de forma paralela pero con algunos puntos de integración.

Orden sugerido:

1. Backend define los endpoints y contratos de la API.
2. Infraestructura crea los contenedores para ejecutar el sistema.
3. Observabilidad instrumenta la API y configura el monitoreo.

Finalmente se integran todos los componentes mediante docker-compose.

6. Entregables de la Fase 1

Para la entrega se debe disponer de:

1. Repositorio Git con el código del proyecto.

2. README con instrucciones para ejecutar el sistema.
 3. Documentos ADR con decisiones de arquitectura.
 4. Servicio accesible para evaluación.
 5. Video demo de aproximadamente 5 minutos mostrando:
 - funcionamiento de la API
 - documentación Swagger
 - monitoreo del sistema
 - ejecución mediante Docker.
-

7. Objetivo del equipo

El objetivo del equipo es que **todos los integrantes comprendan el funcionamiento completo del sistema**, incluso si cada uno tiene una responsabilidad principal.

Esto permitirá:

- responder preguntas técnicas del equipo docente
 - justificar decisiones de arquitectura
 - demostrar un trabajo colaborativo equilibrado.
-